

Descriptive Agent Foundations

What real agents are made of, and why those structures keep getting built

Lecture notes, Agent Foundations module
Iliad Intensive

June 2026

Abstract

There are two ways to ask what an agent is. One asks what an ideal, perfectly rational agent ought to look like, and answers with the coherence theorems: anything that pursues goals without working against itself can be described, from outside, as maximizing expected utility. The other forgets the ideal and looks at the world, which is full of things we are happy to call agents (bacteria, animals, trading firms, trained neural networks) and full of things we are not, and asks what the actual machinery of the real agents is and why that machinery, rather than some other, keeps being produced. These notes develop the second program, assuming no background in agent foundations, probability modeling, or computer science; every idea (a causal diagram, a hidden variable, the trick of reusing stored answers, the notion of a function’s input and output types) is built up from a concrete example before it is named. The organizing tool is the selection theorem, a result that connects a pressure (natural selection, machine learning, market competition) to the kind of internal structure that pressure produces. We first say what “the kind of structure” even means, the type signature of an agent, and why the coherence theorems are best read as one early, badly assumption-laden selection theorem. We then give a concrete and precise picture of the structure we actually expect a realistic agent embedded in the world to have, which looks nothing like the textbook image of a giant table of probabilities over every possible world: it is a sparse web of cause-and-effect relationships built around hidden summary quantities, evaluated lazily so that almost none of it is ever computed, with the agent’s goals attached to the hidden quantities rather than to raw sensory input. We close with the wish list of selection theorems the program hopes to prove, and the pivotal observation that what selection can actually reach is not the highest design but the design surrounded by other decent designs. Throughout, the contrast with the ideal-agent program is the point: the coherence theorems describe how an agent can be summarized; selection theorems try to describe what an agent is, and why.

Contents

1	Two ways to ask what an agent is	2
2	What is “the structure” of an agent?	2
3	Selection theorems	3
3.1	What a selection theorem is	3
3.2	The coherence theorems as a (flawed) selection theorem	3
3.3	Other selection theorems we already have	4
3.4	The research program	4
4	How we picture a real agent	4
4.1	A web of causes, not a table of worlds	5
4.2	Updates that travel only where they matter	5
4.3	Choosing without computing everything	6
4.4	Goals attached to hidden quantities	6
4.5	Reusing stored answers, and the price of doing so	6

5	What selection theorems we want, and what selection can reach	7
5.1	Reachable optima, not merely high ones	7
5.2	The wish list	7
5.3	Why this matters for AI safety	8
6	Takeaways	8
	Sources	9

1 Two ways to ask what an agent is

There are two directions of approach to the question of what an agent is, and the study of agents needs both.

The first direction starts from an ideal. Imagine a perfectly rational decision-maker, ask what consistency forces upon it, and out come the coherence theorems: a non-self-defeating goal-pursuer can be described, from the outside, as if it assigned numerical values to outcomes, assigned probabilities to uncertain possibilities, and chose so as to maximize the probability-weighted value. This is powerful and it holds whatever the agent is built from, which is exactly why it says nothing about what the agent is built from. It characterizes the shadow an effective agent casts on its behavior, not the object casting the shadow.

The second direction starts from the world. As far as we can tell, physical reality at the bottom is something like a vast web of small cause-and-effect interactions, with no agents written into the base layer; it does not come labeled with goals, beliefs, or decisions, and contains no special ingredient marked “mind.” And yet, reliably, regions of this agent-free substrate organize themselves into things that have goals, beliefs, and decisions. Bacteria swim up nutrient gradients, animals build models of their surroundings, firms chase profit, trained networks reduce their loss in ways that look more and more like planning. The descriptive program asks: handed an arbitrary physical system, from a bacterium to a neural network to a future AI, how would we locate its goals, its picture of the world, and its way of deciding? And, going deeper, why do agents with particular structures arise at all, rather than some other structure or none?

What marks this program is that it works from the bottom up. Rather than beginning with ideal principles and asking what a rational agent should do, it begins with facts about the world (that things get selected for performing well, that real reasoners have limited time and memory, that real environments are noisy and built of reusable parts) and asks what kinds of agent-like structure those facts tend to produce. The central formal object is the *selection theorem*, which is to the descriptive program what the coherence theorem is to the ideal one: a result tying a pressure to the structure it favors.

We proceed as follows. Section 2 explains what we even mean by “the structure” of an agent, the notion of a type signature, with the bit of vocabulary it needs built from scratch. Section 3 defines selection theorems, reads the coherence theorems as a first and flawed example, lists other known ones, and lays out the program for finding more. Section 4 is the heart of the notes: a precise, from-scratch picture of the structure a realistic embedded agent actually has, where every mechanism (the cause-and-effect web, the hidden summary variables, the laziness that leaves most of the model uncomputed, the reuse of stored answers) is explained as though you have met none of them before. Section 5 surveys the selection theorems the program wants and the central idea of reachable optima. Section 6 draws the morals and the connections to the rest of the module. The only thing assumed is comfort with the idea of probability as a degree of belief.

2 What is “the structure” of an agent?

Before we can ask what structure gets selected for, we need to say what structure *is*, precisely enough that it could be the conclusion of a theorem. The useful notion borrows an idea from the description of any process that takes things in and puts things out. Think of a recipe, or a machine, or a mathematical function: before knowing how it works inside, you can describe it by what it consumes and what it produces. A bread recipe consumes flour and water and produces a loaf; we can write that down, “flour and water become bread,” without saying a word about kneading. This shape, what-goes-in and what-comes-out, is what we will call a *type signature*, and an agent has one.

Definition 2.1 (Type signature of an agent). To give the *type signature* of a class of agents is to answer three questions. First, *representation*: what are the agent’s parts (its picture of the world, its goals, its way of deciding), and as what kind of object is each part stored? Second, *interfaces*: what does each part take in and put out, and how do the parts feed into one another and into the surrounding world? Third, *embedding*: how does this tidy abstract description sit inside the messy low-level physical system, the cells or the circuitry, that actually realizes the agent?

A compact piece of shorthand captures the functional shape we expect, and it is worth unpacking slowly because the notation is unfamiliar to many readers and trivial once seen. Write “ A to B ” for the type of thing that takes an A and yields a B , the way a recipe is a “flour to bread” thing. Let A stand for the agent’s available actions and B for the outcomes those actions produce. Then “ A to B ” is the type of a *model*: something that, handed a contemplated action, returns the outcome it would lead to. And an agent is one step up: it is a thing that takes a model (an “ A to B ”) and produces an action (A), presumably an action its model rates well. So the agent’s type is “(A to B) to A .” This is only a sketch of a full signature, leaving out uncertainty and observation and much else, but it makes vivid what the program is after: not the agent’s behavior, but the shape of its insides, and how that shape is realized in physical stuff. The deep question is which regions of the agent-free physical web “round off” into something with this agent-shaped type, and why.

The three questions are not idle taxonomy. They are precisely what we must answer to do anything with a powerful AI we did not hand-design. What stores a goal: a utility function, or something else, and does it take whole states of the world as input or only the observable ones? What stores the world-picture, and how does it correspond to the actual world? Does a given system even *have* a cleanly separable goal and world-picture, or are they tangled together past separation? Does it optimize inside a model of the world or directly against the world? And how do any of these abstract parts map onto the weights of a network or the chemistry of a cell? A selection theorem earns its keep exactly when it pins down a piece of this signature for the agents some pressure produces.

3 Selection theorems

3.1 What a selection theorem is

Definition 3.1 (Selection theorem). A *selection theorem* tells us something about the type signature, or the properties, of the agents that get selected for, by some optimization process such as natural selection, machine-learning training, or market competition, within some broad class of environments.

A selection theorem need not mention selection out loud. It might instead describe the best-performing designs for some objective, or the designs that are best across a wide range of conditions, and leave us to note that selection processes climb toward such designs. What makes it a selection theorem is that it links an *environmental pressure* to an *agent structure*, saying “in environments like these, the things that do well are built like that.” The appeal of the framing is that it breaks the vague question “what is an agent” into many concrete, attackable subquestions sharing a common shape, so different researchers can chip at different pieces without stepping on each other, and a newcomer needs little preparation to contribute one more brick.

3.2 The coherence theorems as a (flawed) selection theorem

The quickest way to understand the genre is to notice we already have one specimen, and that examining its defects shows what a better one would need. The coherence theorems are a selection theorem. Read as one, they say: among agents under a resource pressure (do not squander money, or whatever plays money’s role), the ones that cannot be drained have a definite type signature. Its *representation* is a utility function paired with a probability distribution. Its *interfaces* are that the utility and the probabilities take bet-outcomes as inputs, the agent outputs whichever action maximizes expected value, and observations revise the probabilities. Its *embedding* is purely behavioral: the utility and the probabilities are tied to the physical system only by the demand that the system’s choices match what they prescribe.

This is genuinely a selection theorem, and it is also a cautionary one, because its assumptions are visibly too strong for any real agent. To reach the clean conclusion, it must assume the agent’s preferences are *fixed* (they never change), *complete* (it has a definite preference between every possible pair of options), and *path-independent* (only the present options matter, never the history of how you got here). Real agents, humans most obviously, break all three: tastes drift, countless pairs of options are simply not comparable, and what we will accept depends heavily on where we started. So the coherence theorems, viewed as a selection theorem, have the right *form* (a pressure in, a structure out) but the wrong *assumptions and conclusion* for the agents we care about. The descriptive program’s bet is that better selection theorems, founded on more realistic assumptions about changing internal state, incomplete preferences, and physical embedding, would yield more faithful type signatures, and that those are exactly what we need to make sense of human values, of optimizers that arise inside other optimizers, and of values that drift, which are central problems of AI safety.

3.3 Other selection theorems we already have

The coherence theorems are not alone, and the existing examples show how varied the genre is. The *Good Regulator theorem*, and Wentworth’s repaired *Gooder Regulator* version, describe conditions under which an agent that successfully keeps some system under control must contain a model of that system, a result directly about when world-models are selected for. The *Kelly criterion* shows that an agent repeatedly wagering a fraction of its wealth, selected to dominate in the long run, ends up behaving as though it maximizes expected logarithm of wealth, a case where one pressure (long-run growth) fixes a specific betting rule. The *subagents argument* shows that once an agent carries internal state, the coherence conclusion changes shape: such an agent’s revealed goals look like a committee of several utility functions seeking arrangements that none of them can improve on, rather than a single utility being maximized. Each pins down some slice of the type signature, and together they sketch what a mature theory would be: a library of results connecting recognizable pressures to recognizable pieces of agent structure.

3.4 The research program

How does one find new selection theorems? The methodology runs in two complementary directions. One can start from *observation* of evolved and trained agents across biology, economics, and machine learning, build up intuitions about what structure shows up under what conditions, and try to promote a specific observation into a general theorem. (A standing example: organisms fixed in place, like plants and sponges, rarely evolve brains, while mobile ones often do; can that become a general theorem relating the worth of a control system to whether the agent can act on what it learns?) Or one can work backward, starting either from a structure that looks sensible, based on how humans are built, and asking what pressure would produce it, or from a pressure and asking what structure it should yield. The other direction is incremental work on the theorems we have: apply one to a real system and fix what breaks (the subagents idea was born exactly this way, from applying coherence theorems to financial markets and watching them fail), strengthen a result, patch a hole as the Gooder Regulator theorem did, unify several results, or strip a domain-specific theorem down to its minimal assumptions so it covers more. The claim animating the whole effort is ambitious and stated plainly: sufficiently good selection theorems would get us most of the way to the hardest parts of AI safety, because they would tell us what the systems we are about to build are actually made of.

4 How we picture a real agent

To make the program concrete we need a candidate for the type signature itself, not the idealized one the coherence theorems hand us but the one a realistic agent embedded in the world would actually have. Here the descriptive program parts ways sharply with the textbook, and the picture deserves to be developed carefully and from scratch, because it is exactly what the hoped-for selection theorems are trying to derive.

Begin with the textbook image, so we can see what is wrong with it as a literal mechanism. A Bayesian agent, as usually defined, considers every possible way the entire world could be, assigns each a probability, and on each new observation revises the whole assignment, computing what it wants by summing over all those possibilities. We often *define* a Bayesian agent as one whose outward behavior matches this

prescription. But matching the behavior does not require carrying out the prescription literally, and carrying it out literally is impossible for anything embedded in the world, because the number of distinct ways the world could be is vastly larger than any physical memory could list, let alone update entry by entry. So the real question is what an agent that behaves Bayesian, but is built to fit inside the world, looks like on the inside. The answer of Wentworth and Lorell has several interlocking parts, and we take them one at a time.

4.1 A web of causes, not a table of worlds

The first correction is how the world-picture is stored. Instead of one probability for each complete way the world could be, of which there are astronomically many, the agent stores a *causal diagram*: a collection of variables (the temperature, whether it is raining, whether the streets are wet) drawn as dots, with an arrow from one variable to another when the first directly influences the second (rain to wet streets). Such a diagram is also called a Bayes net or a causal model. What it buys you is compactness. To describe how the whole collection of variables behaves jointly, you do not store a number for every global combination, which would be a quantity that explodes exponentially as you add variables. You store, for each variable, only how it depends on the handful of variables that point directly at it (how “wet streets” depends on “rain” and “street cleaning,” and nothing else), and the global picture is reconstructed from these local pieces. The storage grows roughly in proportion to the number of variables, not exponentially, because each variable contributes only its own little local rule. This is the difference between writing down a town’s road map, which lists each intersection and the roads meeting at it, and writing down a separate entry for every possible route through the entire town.

The second correction makes the picture even more compact, and it is the one most worth slowing down on. The local rules can themselves *refer back to the same model on a smaller input*, so that a finite description stands in for an unboundedly large diagram. Consider how one defines the factorial, the product $n \times (n-1) \times \cdots \times 1$. You need not write out a separate diagram for each n . You write a single self-referential rule: the factorial of n is n times the factorial of $n-1$, with the factorial of 0 being 1. That one finite rule describes the entire infinite family of computations, of every size, because it mentions itself on a smaller input. A realistic world-model is structured the same way: it is less like a fixed picture and more like a moderately-sized set of rules that call themselves and each other, so that even an effectively unbounded cause-and-effect structure (the unrolled steps of some long process) has a short description. The world-model is a moderately-sized program with a lot of self-reference, not a flat table.

4.2 Updates that travel only where they matter

The next correction is about computation, specifically what happens when the agent learns something. In the textbook picture, every observation forces a revision of the entire belief table. In the realistic picture, almost nothing gets revised, because almost nothing is relevant. Here is the precise mechanism. In a causal diagram, learning the value of one variable changes your beliefs about its neighbors, which changes your beliefs about *their* neighbors, and so on; the change propagates outward along the arrows like a rumor passing person to person. This propagation is carried out by what is called *message passing*: each variable, upon updating, sends each neighbor a “message” summarizing how much that neighbor’s beliefs should shift, the neighbor incorporates it and passes along its own messages, and the process ripples through the diagram. The realistic claim is that these messages shrink toward nothing as they travel, because most variables are connected to the observation only through long chains of weak influence, so the update that finally reaches a distant variable is negligible and can be ignored. You see a bird fly past your office window. That observation is, in principle, faintly connected through the world’s causal web to the price of gasoline, but only through such a long and tenuous chain that the message arriving at “gasoline price” is effectively zero. A sane reasoner does not propagate the bird to its beliefs about gasoline at all. The update converges after touching a small neighborhood of relevant variables and leaves the vast remainder of the model untouched, which is what makes updating affordable.

4.3 Choosing without computing everything

The same laziness governs decisions, and the precise statement is sharper than it first sounds: an agent maximizing expected utility never actually has to compute an expected utility. Suppose you are choosing your lunch between a lamb dish and a burrito. The textbook recipe says to compute the total expected utility of your entire future given the lamb, compute it again given the burrito, and compare. But your entire future includes your career, your friendships, the fate of distant nations, almost none of which depends on today's lunch. What you actually need is only the *difference* the two options make, and that difference is exactly zero for the overwhelming majority of the world, since both lunches leave your career and the distant nations the same. So when you propagate "lamb versus burrito" through your causal model, the contribution to the utility difference dies out almost immediately, just as the bird's message died out before reaching gasoline, and you end up computing only the few things that genuinely differ between the options (taste, price, how full you will feel). You compare options by computing differences, the differences vanish across nearly all the model, and you are spared ever summing utility over the whole world. Maximizing expected utility, done by a real agent, queries almost none of its own model.

4.4 Goals attached to hidden quantities

The next correction concerns what the agent's goals are even *about*. Real models are organized around *latent variables*, which are hidden quantities the agent never observes directly but posits in order to explain the things it does observe. A child notices that bark, leaves, roots, and a certain branching shape tend to occur together, and posits a single hidden thing, "tree," that explains the cluster; asked why things with bark also have roots, the answer is "because they are trees." The latent "tree" is not itself seen; it is invented to summarize and account for the correlations among what is seen. These hidden variables need not correspond to anything real (human models have posited spirits and other latents that turned out empty), but in practice most of them do correspond to real structure, and explaining why is the puzzle of the natural-abstraction program discussed below. What matters most for safety is where the goals live. The inputs to a real agent's goals are typically these hidden, latent quantities, not the raw observations. You want your spouse to *actually be happy*, which is a hidden state of another mind, not merely to *look happy to you*, which is the raw observation. The difference between those two goals is precisely the difference between a goal defined over a latent variable and a goal defined over an observation, and it is the difference between a marriage and a deception.

4.5 Reusing stored answers, and the price of doing so

The final correction explains why a realistic agent is, much of the time, slightly incoherent, and why fixing that takes effort. For speed, the agent stores the results of computations it has done before and reuses them rather than redoing them, a practice called *caching*; more specifically it stores the worked-out value of being in various intermediate situations, a technique called dynamic programming (solve a big problem once by storing the solutions to its sub-situations and assembling them). The stored values are supposed to satisfy a consistency condition, called the *Bellman equation*, which just says that the stored value of a situation should equal the value of the best immediate move plus the stored value of wherever that move leads; the worth of being here is the worth of the best step plus the worth of being there. The trouble is that stored values go stale. As the agent learns new things, cached values computed earlier can come to violate this consistency, disagreeing with each other or with the agent's actual goals. So a realistically embedded agent is locally incoherent a good deal of the time, holding stored judgments that no longer fit together. Detecting such a clash is local and cheap (check whether neighboring stored values satisfy the Bellman relation), but repairing it can be expensive: the correction has to propagate through the model, updating value after value, which is to say the agent has to stop and think things through. This is a candidate explanation for why humans, otherwise well described as Bayesian, exhibit characteristic non-Bayesian quirks and feel genuine mental effort when forced to reconcile beliefs that have come into conflict; they are repairing stale caches.

Put the corrections together and the working hypothesis for the type signature of a real agent comes into focus: a self-referential web of cause-and-effect relationships, organized around hidden summary variables, evaluated so lazily that almost none of it is ever computed on any given decision, with goals attached to the hidden variables and a constant background chore of keeping reused answers consistent. The agent appearing

as a variable inside its own model, its self-representation, fits naturally into this framework but is its least developed corner and remains open. The selection theorems of the next section are, in large part, attempts to show that pressure plus realistic environments *produces* exactly these features.

5 What selection theorems we want, and what selection can reach

Granting that target picture, which selection theorems would establish that it is what gets selected for, and under what assumptions? One methodological idea reorganizes the whole search, and it is the single most important point in this section, so it comes first.

5.1 Reachable optima, not merely high ones

The selection theorems we already have mostly describe *optimal* agents: be a perfectly coherent expected utility maximizer or be beaten. But real selection processes do not find perfect optima, and insisting on optimality both overstates the conclusion and misses how selection actually works. Two weaker notions are what matter.

A design is a *broad* optimum if the designs near it, the ones reachable by small changes, are also decent. This matters because a selection process can only *get to* a design by passing through its neighbors. However high the performance of some isolated design, if everything immediately around it in the space of possible designs performs terribly, then neither natural selection (which moves by small mutations) nor machine-learning training (which moves by small adjustments) has any path to it; there is no sequence of small uphill steps leading there. So the designs selection actually finds sit on broad rises, not lonely spikes. A design is a *robust* optimum if it stays good as the environment varies rather than being finely tuned to one exact setting; this matters because real selection samples a varying environment only finitely, so noise in the sampling pushes toward designs that tolerate variation.

The reframing has bite because broad and robust optima are exactly the ones for which the desirable structural features plausibly emerge. A modular design, one built of semi-independent parts, tends to sit on a broad rise precisely because its parts can each be varied a little without the whole collapsing. So the program’s wager is that selecting for breadth and robustness, not merely height, is what produces agent-shaped structure in the first place. (Wentworth sketches a tentative mathematical version, that a design’s peak is broad when the parts of the design interact only weakly, while candidly flagging that a sign error could invert the argument; the qualitative moral stands regardless.)

5.2 The wish list

With reachability as the lens, the conjectured selection theorems form a coherent slate, each the hypothesis that some realistic pressure selects for one feature of the type signature of Section 4.

Conjecture 5.1 (Modularity). Agents selected in environments whose demands vary in a structured, piece-by-piece way come to have *modular* internal structure, built of semi-separable parts. Simulations already show that varying the objective modularly tends to produce modular solutions, which suggests it is robustness to a changing environment, rather than raw efficiency, that drives modularity.

Conjecture 5.2 (Natural abstractions). Agents in sufficiently rich environments come to use the same high-level concepts, the hidden summary variables of Section 4. The underlying idea, the natural-abstraction hypothesis, is that almost all of the information about a chunk of the world that is still relevant far away gets compressed into a small high-level summary, and agents learn to think in terms of those summaries, which is why distinct agents converge on shared concepts like “tree” or “temperature.”

Conjecture 5.3 (Goals over hidden variables). The goals of selected agents take *hidden, latent quantities* as their inputs, rather than the bet-able outcomes the coherence theorems assume. This is the structural repair for the gap noted in Section 3: practitioners already find themselves writing goals over unobservable states of the world, and selection should favor goals defined over the natural summary variables an agent already uses to model the world.

Conjecture 5.4 (A committee of goals). The goals of selected agents decompose into a *set* of utilities, one per internal subagent, rather than a single utility function, generalizing the subagents argument. Suggestive evidence includes the way many moral insights feel like deals that make several internal parties, valuing different things, all better off at once.

Conjecture 5.5 (Optimizers inside optimizers). As an environment grows more complex and variable, the systems selected in it increasingly contain their *own* optimization processes; such an inner optimizer produced by an outer optimization process is called a *mesa-optimizer*. The logic is a make-or-buy decision about *when* to optimize. Some optimizing can be done in advance, by the outer process that shapes the agent; but when the best action varies a lot in ways that can only be worked out from information available at the moment of acting, it pays to defer the optimizing to that moment, inside the agent. That deferred, on-the-spot optimization is exactly what a *general-purpose search* is, and it is the core of agency. This reframes inner optimizers not as a freak malfunction but as a likely product of scaling, which is just why a mesa-optimizer pursuing the wrong goal is a serious safety worry.

Conjecture 5.6 (Separate modules for the separate jobs). In large agents selected in sufficiently complex environments, the internal structure has *distinct parts* for the world-model, the goal, and the search or optimization process, the clean separation the type signature of Section 2 takes for granted.

Each conjecture comes with environmental assumptions of a recognizable kind: that the environment’s structure varies in a modular way over time or space, that it is complex and variable enough to make on-the-spot optimization worth the trouble, that selection samples it finitely so that robustness is rewarded, and that its low-level structure admits compressing summaries in the first place.

5.3 Why this matters for AI safety

The payoff of the wish list is that it speaks directly to the questions safety most needs answered about systems we train rather than design. If selection favors modular agents with separable, explicit world-models, goals, and search processes, then large trained systems might carry interpretable structure: goal content one could read out before deployment, parts one could test in isolation, decision chains one could trace. The same reachability argument that drives the breadth story suggests why we should expect trained systems to have structure at all, with the pressure toward it strengthening as data, task complexity, and scale grow. The flip side is the register of risks the conjectures generate. Optimizers-inside-optimizers predicts inner optimizers that may chase proxy goals diverging from the training objective. Goals-over-hidden-variables together with concept convergence implies that alignment is tractable to the extent that a system’s hidden summary variables line up with ours, with specification gaming and value mismatch the failure mode when they do not. And modular agents with explicit world-models are exactly the ones structurally able to model their own training process and feign compliance. The descriptive program does not by itself deliver safety; it aims to tell us which of these worlds we are in.

6 Takeaways

The cleanest way to hold descriptive agent foundations in mind is as the mechanistic counterpart to the representational results of the ideal-agent program. The complete class theorem and its relatives are *representational*: they show that any undominated agent *can be described* as an expected utility maximizer, while saying nothing about how it is built. Selection theorems aim to be *mechanistic*: they ask what agents actually *are*, what their parts are and how those parts are wired, and what pressures bring such structures into being. The two are complementary halves of one question. The representational result fixes the target shape that effective behavior converges toward; the mechanistic program asks which physical pressures produce that shape, what an agent is made of when they do, and what physical quantity plays the role of the measuring stick the coherence arguments quietly assumed.

This is why the descriptive program threads through the rest of the module. It inherits from consequentialist foundations the open problem of the measuring stick, the question of identifying, from physics alone and with no agency assumed in advance, what counts as a resource and hence where in the world optimization is happening. It shares with the optimization-and-thermodynamics material the bottom-up

stance that starts from physical constraints rather than ideal axioms, since a selection environment is, after all, a physical environment, and physics bounds what it can demand and reward. Its picture of a real agent, a self-referential web of causes around hidden variables, lazily evaluated, with goals over those hidden variables, is a concrete hypothesis about what a world-model really is, and its conjecture that on-the-spot search gets selected for when optimization is best deferred is a concrete hypothesis about what general-purpose search really is. None of the central conjectures is yet a theorem, and the program is candid that its current best specimen, the coherence theorems, rests on assumptions that fail for the agents we care about. That candor is its strength: it knows exactly which assumptions to attack, and it is wagering that getting the descriptive structure of real agents right is most of what it would take to understand, and to align, the agents we are on the verge of building.

Sources

- John Wentworth, *Selection Theorems: A Program For Understanding Agents* (read in full). <https://www.lesswrong.com/posts/G2Lne2Fi7Qra5Lbuf/selection-theorems-a-program-for-understanding-agents>
- John Wentworth and David Lorell, *How We Picture Bayesian Agents* (read in full). <https://www.lesswrong.com/posts/TiBsZ9beNqDHEvXt4/how-we-picture-bayesian-agents>
- John Wentworth, *What Selection Theorems Do We Expect/Want?* (read in full). <https://www.lesswrong.com/posts/RuDD3aQWLDSb4eTXP/what-selection-theorems-do-we-expect-want>